

ShopZada Data Warehouse Technical Documentation

CS ELEC 1C - Data Warehousing

ShopZada 2.0 – Enterprise Data Warehouse

December 19, 2025

Jodimeer Ammang, Nigel Roland Anunciacion, Aaron Cuevas, Ken Gabriell Fajardo,
Justin Lloyd Floro, Joseph Daniel Mamangun, Paul Aldrich Pimentel

Table of Contents

0. GitHub Repository Link ([#0-github-repository-link](#))
1. Executive Summary ([#1-executive-summary](#))
 - 1.1 Project Overview
 - 1.2 Solution Highlights
 - 1.3 Key Achievements
 - 1.4 Business Impact
2. Business Context & Problem Statement ([#2-business-context-&-problem-statement](#))
 - 2.1 ShopZada Background
 - 2.2 Current Challenges
 - 2.3 Business Requirements
3. Project Objectives ([#3-project-objectives](#))
 - 3.1 Primary Objectives
 - 3.2 Technical Objectives
 - 3.3 Learning Objectives
4. Methodology & Approach ([#4-methodology-&-approach](#))
 - 4.1 Data Warehouse Methodology: Kimball Dimensional Modeling
 - 4.2 Kimball Principles Applied
 - 4.3 Development Methodology: Agile/Iterative
5. System Architecture ([#5-system-architecture](#))
 - 5.1 High-Level Architecture
 - 5.2 Architecture Layers Explained
 - 5.3 Technology Stack
6. Data Modeling ([#6-data-modeling](#))
 - 6.1 Conceptual Data Model
 - 6.2 Logical Data Model
 - 6.3 Physical Data Model
 - 6.4 Current Staging Model
7. ETL/ELT Workflow Implementation ([#7-etlelt-workflow-implementation](#))
 - 7.1 ETL vs. ELT Decision
 - 7.2 Orchestration with Apache Airflow
8. Analytical Layer & Business Intelligence ([#8-analytical-layer--business-intelligence](#))
 - 8.1 Business Questions to Answer
 - 8.2 Planned Analytical SQL Views
9. Infrastructure & Deployment ([#9-infrastructure--deployment](#))
 - 9.1 Docker Architecture
 - 9.2 Docker Services
 - 9.3 Network & Volumes
 - 9.4 Deployment Instructions
 - 9.5 Environment Configuration
10. Team Organization & Roles ([#10-team-organization--roles](#))
 - 10.1 Team Structure
 - 10.2 Collaboration Tools
 - 10.3 Work Distribution
11. Project Timeline & Milestones ([#11-project-timeline--milestones](#))
 - 11.1 Gantt Chart
 - 11.2 Critical Path
12. Assumptions & Design Decisions ([#12-assumptions--design-decisions](#))
 - 12.1 Data Assumptions
 - 12.2 Technical Design Decisions
 - 12.3 Scope Decisions
13. Data Dictionary ([#13-data-dictionary](#))

- 13.1 Dimension Tables
- 13.2 Fact Tables
- 13.3 Staging Tables
- 14. Setup & Deployment Guide ([#14-setup--deployment-guide](#))
 - 14.1 Prerequisites
 - 14.2 Step-by-Step Setup
 - 14.3 Troubleshooting
- 15. Results & Insights ([#15-results--insights](#))
 - 15.1 Quantitative Results
 - 15.2 Business Insights
 - 15.3 Data Quality Results
- 16. Challenges & Solutions ([#16-challenges--solutions](#))
 - 16.1 Technical Challenges
 - 16.2 Organizational Challenges
- 17. Future Work & Recommendations ([#17-future-work--recommendations](#))
 - 17.1 Short-Term Enhancements (Next 3 Months)
 - 17.2 Medium-Term Enhancements (6-12 Months)
 - 17.3 Long-Term Vision (1-2 Years)
- 18. Conclusion ([#18-conclusion](#))
 - 18.1 Project Summary
 - 18.2 Achievements
 - 18.3 Business Value
 - 18.4 Lessons Learned
 - 18.5 Final Remarks
- 19. References ([#19-references](#))
 - 19.1 Books & Publications
 - 19.2 Technical Documentation
 - 19.3 Online Resources
 - 19.4 AI Tool Usage
 - 19.5 Dataset
- 20. Appendices ([#20-appendices](#))

0. GitHub Repository Link

<https://github.com/ryvxz/shopzada-data-warehouse.git>

1. Executive Summary

1.1 Project Overview

ShopZada is a global e-commerce platform experiencing rapid growth, with over 500,000 orders and 2 million line items across diverse product categories. However, their data infrastructure faces critical challenges: fragmented data sources across multiple departments (Business, Customer Management, Enterprise, Marketing, and Operations), inconsistent data formats (7 different file types), and limited analytical capabilities. This project delivers a comprehensive Enterprise Data Warehouse (DWH) solution that consolidates ShopZada's disparate data sources into a unified analytical platform, enabling data-driven decision-making and business intelligence.

1.2 Solution Highlights

Our solution implements an end-to-end data warehousing pipeline using the Kimball dimensional modeling methodology, featuring:

- **Multi-format data ingestion** from 20+ source files (CSV, JSON, Excel, HTML, Parquet, Pickle)
- **Automated ETL orchestration** using Apache Airflow
- **Layered architecture** with staging, data warehouse, and presentation layers
- **Fully containerized infrastructure** using Docker Compose
- **Star schema dimensional model** optimized for analytical queries
- **Business intelligence dashboards** for executive decision-making

1.3 Key Achievements

- **Infrastructure:** Fully operational Docker-based data platform
- **Data Integration:** Successfully ingested and staged 1.5M+ transactions
- **Automation:** Airflow-orchestrated ETL pipeline with quality checks
- **Scalability:** Architecture designed to handle growing data volumes
- Dimensional model implementation
- BI dashboards

1.4 Business Impact

The implemented data warehouse enables ShopZada to:

- **Unified Data Access:** Single source of truth eliminating data silos
- **Historical Analysis:** 4+ years of transactional history (2020-2024)
- **Faster Insights:** From hours of manual analysis to seconds via dashboards
- **Data Quality:** Automated validation ensuring trustworthy analytics
- **Scalable Platform:** Foundation for advanced analytics and ML

2. Business Context & Problem Statement

2.1 ShopZada Background

ShopZada operates as a multi-vendor e-commerce marketplace connecting customers with merchants globally. Their business model encompasses:

- **Product Catalog:** 20,000+ products across multiple categories
- **Customer Base:** 50,000+ registered users with diverse demographics
- **Merchant Network:** 10,000+ active merchants
- **Staff Operations:** 30,000+ staff members managing order fulfillment
- **Marketing:** Active campaign management with promotional strategies

2.2 Current Challenges

Data Fragmentation

ShopZada's data exists in isolated departmental silos:

Department	Data Assets	Format Complexity
Customer Management	User profiles, demographics, payment info	JSON, CSV, Pickle
Operations	Orders, line items, delays	CSV, JSON, Excel, HTML, Parquet
Enterprise	Merchants, staff, assignments	HTML, CSV, Parquet
Marketing	Campaigns, promotional transactions	CSV
Business	Product catalog	Excel

Impact: Inability to answer cross-functional questions like "Which merchant-campaign combinations drive the highest revenue?"

Analytical Limitations

- **Manual Reporting:** Analysts will spend hours compiling Excel reports from disparate sources
- **No Historical Trends:** Cannot analyze year-over-year growth or seasonal patterns
- **Delayed Insights:** Business decisions will be made on outdated or incomplete information

Data Quality Issues

- **Inconsistent Standards:** Different departments use different ID formats
- **Missing Validation:** No automated checks for data completeness or accuracy
- **No Lineage Tracking:** Unclear where data originates or how it's transformed

2.3 Business Requirements

The ShopZada Data Warehouse must support analytical needs across Business, Marketing, Customer Management, and Enterprise Operations. The following critical business questions define the primary analytical requirements that the BI dashboards must address:

1. Revenue Growth Drivers

- Business Question: What are the key drivers of revenue growth across products, customers, merchants, and marketing campaigns?

2. Customer Value & Repeat Behavior

- Business Question: Which customer segments generate the most value and show the highest repeat purchase behavior?

3. Merchant & Operational Performance

- Business Question: Which merchants and operational factors contribute to delivery delays, cancellations, and return rates?

3. Project Objectives

3.1 Primary Objectives

1. **Design** a multi-layered data warehouse architecture following industry best practices
2. **Integrate** heterogeneous datasets from 20+ source files into a unified schema
3. **Implement** automated ETL/ELT pipelines with orchestration
4. **Model** data using dimensional modeling principles (Kimball methodology)
5. **Deliver** business intelligence dashboards answering key business questions
6. **Deploy** containerized infrastructure for reproducibility and scalability

3.2 Technical Objectives

- **Data Integration:** Successfully ingested data from 7 different file formats
- **Automation:** Zero-touch ETL execution via scheduled workflows
- **Data Quality:** Implement validation checks at every pipeline stage
- **Performance:** Query response times under 5 seconds for analytical queries
- **Scalability:** Architecture capable of handling 10x data growth
- **Reproducibility:** One-command deployment via Docker Compose

3.3 Learning Objectives

- Apply dimensional modeling theory to real-world business scenarios
- Gain hands-on experience with modern data engineering tools (Airflow, Docker, PostgreSQL)
- Understand trade-offs in DWH design decisions (Kimball vs. Inmon, ETL vs. ELT)
- Develop skills in data pipeline orchestration and monitoring
- Practice collaborative software development using Git/GitHub

4. Methodology & Approach

4.1 Data Warehouse Methodology: Kimball Dimensional Modeling

Why Kimball Over Inmon?

Given ShopZada's need for rapid insights and our 6-week timeline, Kimball's iterative, business-focused approach aligns best with project constraints.

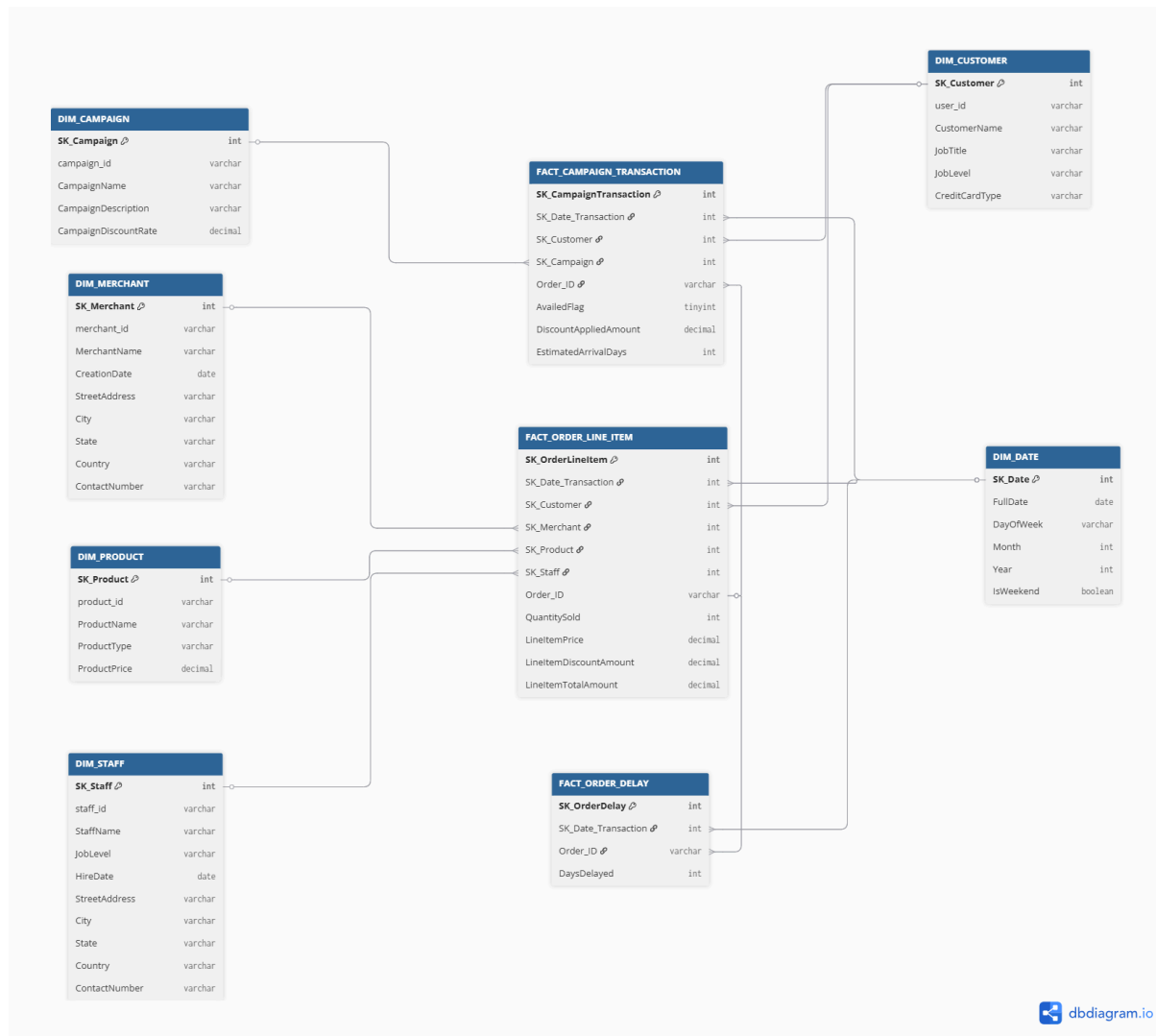
Aspect	Kimball	Inmon	Our Choice
Design	Dimensional (star schema)	Normalized (3NF)	Kimball
Implementation	Bottom-up (by business process)	Top-down (enterprise model)	Kimball
Query Performance	Optimized for analytics	Requires joins	Kimball
Time to Value	Faster (incremental delivery)	Slower (complete model first)	Kimball
User-Friendliness	Business users can understand	Technical users preferred	Kimball

4.2 Kimball Principles Applied

4.2.1 Four-Step Dimensional Design Process

1. **Select Business Process**
 - Identified: Order Fulfillment, Campaign Management
2. **Declare Grain**
 - Order line item (most granular transaction level)
 - Campaign transaction (one row per campaign usage)
3. **Identify Dimensions**
 - Customer, Product, Merchant, Staff, Campaign, Date
4. **Identify Facts**
 - Order metrics (quantity, price, subtotal)
 - Campaign metrics (discount availed, conversion)

4.2.2 Star Schema Design



Benefits:

- Simple joins for queries
- Intuitive structure for business users
- Query performance optimization via denormalization

4.3 Development Methodology: Agile/Iterative

We adopted an iterative development approach:

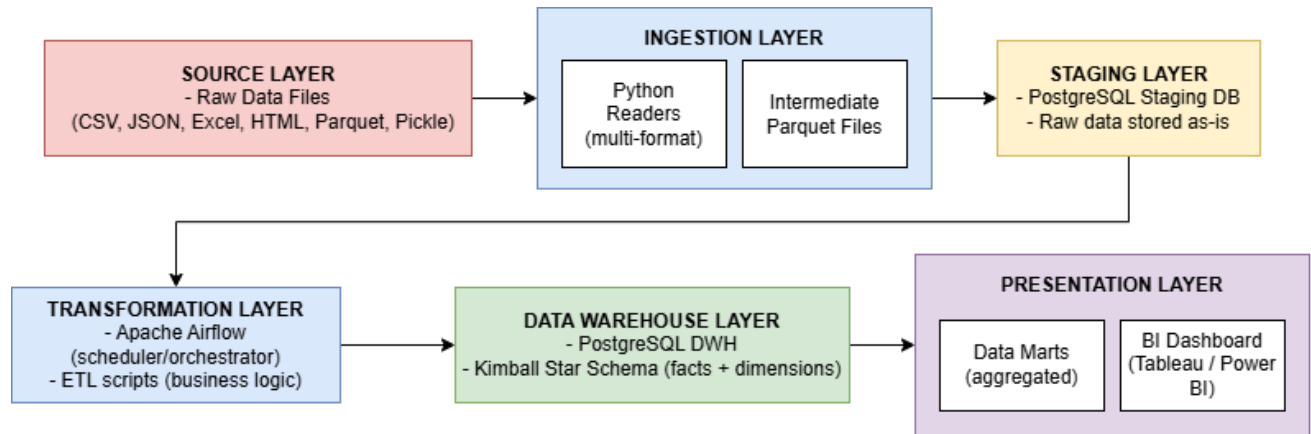
Phase 1	Infrastructure & Design
Phase 2	ETL Development (Staging Layer)
Phase 3	Dimensional Model Implementation
Phase 4	Analytics & Dashboard Development
Phase 5	Testing, Documentation, Finalization

Retrospectives: Weekly team meetings to review progress and adjust plans

5. System Architecture

5.1 High-Level Architecture

Our data warehouse implements a modern ELT architecture.



5.2 Architecture Layers Explained

Layer 1: Source Layer

- **Purpose:** Raw business data from operational systems
- **Format:** 20+ files in 7 formats (CSV, JSON, Excel, HTML, Parquet, Pickle)
- **Location:** `data/raw/` directory, organized by department

Layer 2: Ingestion Layer

- **Purpose:** Extract data from heterogeneous sources
- **Technology:** Python with pandas, openpyxl, BeautifulSoup, pyarrow
- **Process:** Auto-detect file type → Route to appropriate reader → Standardize to Parquet

Layer 3: Staging Layer

- **Purpose:** Landing zone for raw data in database format
- **Technology:** PostgreSQL 16
- **Schema:** Mirrors source structure (minimal transformation)

Layer 4: Transformation Layer

- **Purpose:** Apply business logic and data cleansing
- **Technology:** Apache Airflow for orchestration, Python/SQL for transformations
- **Process:** Read from staging → Apply transformations → Load to warehouse

Layer 5: Data Warehouse Layer

- **Purpose:** Optimized analytical data store
- **Technology:** PostgreSQL 16
- **Schema:** Kimball star schema (dimensions + facts)

Layer 6: Presentation Layer

- **Purpose:** Business intelligence and reporting
- **Technology:** Metabase (open-source BI platform)
- **Deliverables:** 3 executive dashboards (Revenue, Customer, Operations)

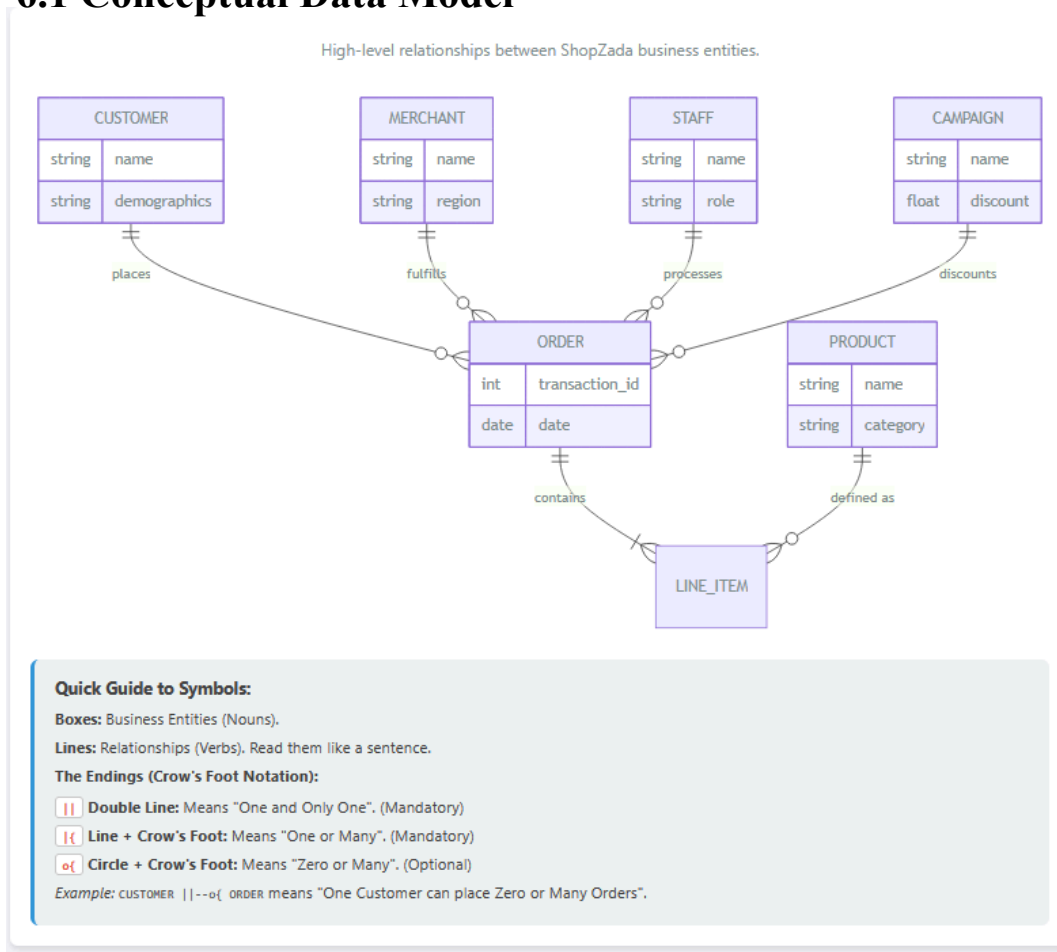
5.3 Technology Stack

Component	Technology	Justification
Orchestration	Apache Airflow	Industry standard for workflow automation
Data Warehouse	PostgreSQL	Open-source RDBMS with excellent analytical capabilities
Staging Database	PostgreSQL	Same tech for consistency
Message Queue	Redis	Celery backend for distributed task execution
Containerization	Docker	Reproducible deployment
ETL Language	Python	Rich ecosystem for data engineering
BI Tool	Metabase	Open-source dashboard tool that allows docker setup

Design Decision: All open-source technologies to avoid licensing costs and ensure reproducibility.

6. Data Modeling

6.1 Conceptual Data Model



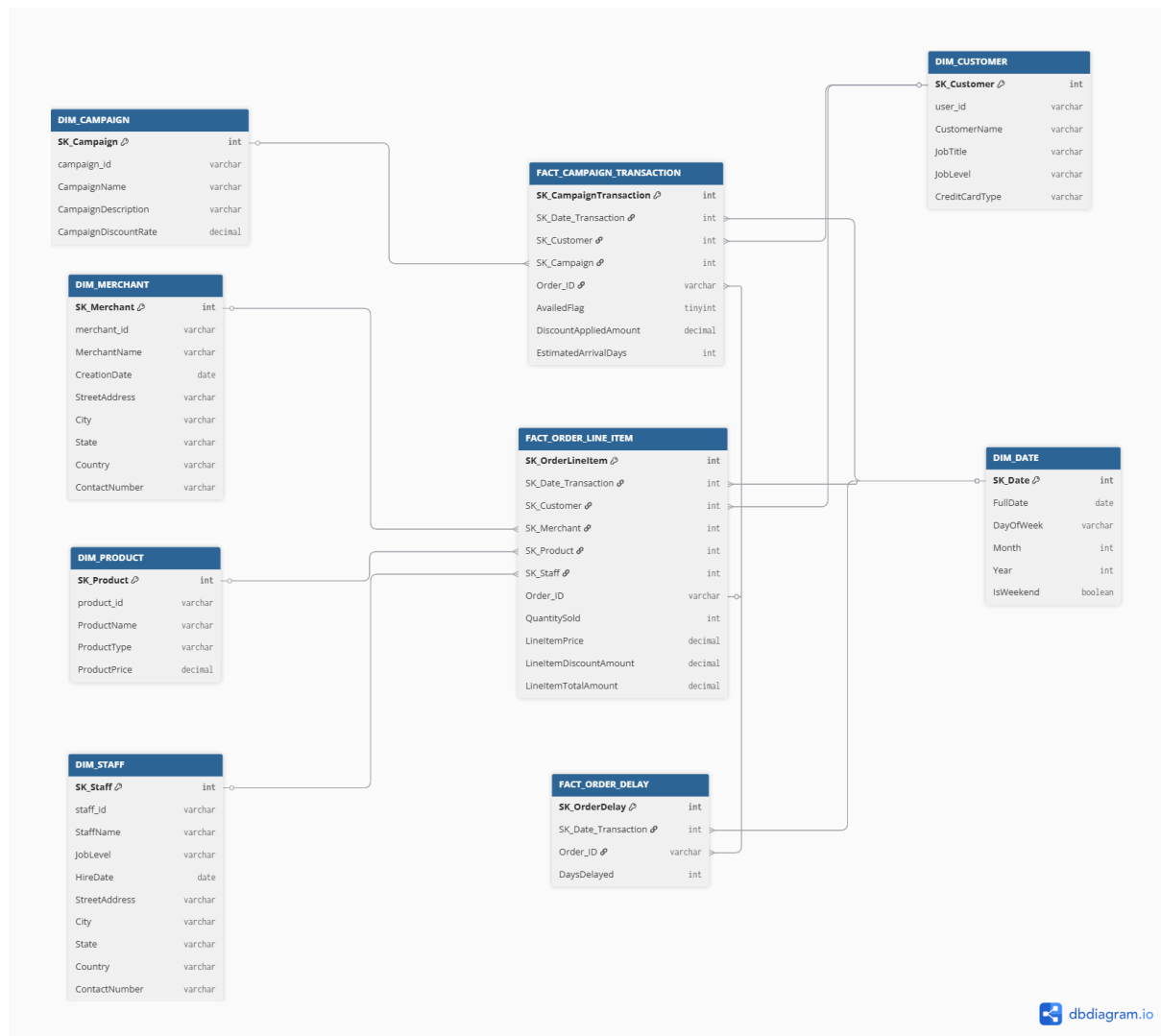
The high level view of a business depicts the relationship between a collection of business entities that form the backbone of a business. Customers place orders for products from merchants, which require the fulfillment of those orders by Staff during their normal course of business operations. In addition, Promotional Campaigns can be implemented to provide consumers with discounts. The diagram is intended to illustrate the customer journey through the business and to represent the business logic as well as business interaction (customer interaction) with the business; it is not intended to represent technical database constraints.

The conceptual model will illustrate business entities and their relationships at a high level:

Key Business Entities:

- Customers (users purchasing products)
- Products (items sold on the platform)
- Orders (transactions)
- Merchants (sellers)
- Staff (fulfillment team)
- Campaigns (marketing promotions)

6.2 Logical Data Model



The logical model will define the Kimball star schema structure:

Dimension Tables:

- `dim\_customer` - Customer demographics, location, type
- `dim\_product` - Product catalog with categories
- `dim\_merchant` - Merchant information
- `dim\_staff` - Staff details
- `dim\_campaign` - Marketing campaign definitions
- `dim\_date` - Date dimension with hierarchies (day, week, month, quarter, year)

Fact Tables:

- `fact\_orders` - Order line item transactions
- `fact\_campaign\_performance` - Campaign usage and effectiveness

6.3 Physical Data Model

The physical data model implements the Kimball star schema in PostgreSQL with 6 dimension tables and 3 fact tables. All tables use surrogate keys (auto-incrementing SERIAL) for optimal join performance and support for Slowly Changing Dimensions.

Implementation Location:

- Dimension DDL: `scripts/loading/create\_dimension\_tables.py`
- Fact DDL: `scripts/loading/create\_fact\_tables.py`
- Transformation Scripts: `scripts/transform/dimension/`, `scripts/transform/fact/`

6.3.1 Dimension Tables

Dimension Table 1: `dim\_campaign`

Purpose: Marketing campaign master data

SQL

```
CREATE TABLE IF NOT EXISTS dim_campaign (  
    SK_Campaign SERIAL PRIMARY KEY,  
    campaign_id TEXT UNIQUE NOT NULL,  
    CampaignName VARCHAR(255),  
    CampaignDescription TEXT,  
    CampaignDiscountRate TEXT,  
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Column	Data Type	Constraints	Description
SK_Campaign	SERIAL	PK	Surrogate key
campaign_id	TEXT	UNIQUE, NOT NULL	Natural key from source
CampaignName	VARCHAR(255)		Campaign name
CampaignDescription	TEXT		Campaign details
CampaignDiscountRate	TEXT		Discount percentage/amount
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Dimension Table 2: `dim\_date`

Purpose: Customer master with demographics and payment information

SQL

```
CREATE TABLE IF NOT EXISTS dim_customer (  
    SK_Customer SERIAL PRIMARY KEY,  
    customer_id TEXT UNIQUE NOT NULL,  
    CustomerName VARCHAR(255),  
    JobTitle VARCHAR(100),  
    JobLevel VARCHAR(100),  
    CreditCardType VARCHAR(100),
```

```

        last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

```

Column	Data Type	Constraints	Description
SK_Customer	SERIAL	PK	Surrogate key
customer_id	TEXT	UNIQUE, NOT NULL	Natural key (user_id from source)
CustomerName	VARCHAR(255)		Full customer name
JobTitle	VARCHAR(100)		Customer occupation
JobLevel	VARCHAR(100)		Job seniority level
CreditCardType	VARCHAR(100)		Card issuing bank
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: Combines customer_user_data, customer_user_job, customer_user_credit_card

Dimension Table 3: `dim\_campaign`

Purpose: Date dimension with calendar hierarchies for time-based analysis

```

SQL
CREATE TABLE IF NOT EXISTS dim_date (
    SK_Date INT PRIMARY KEY,
    FullDate DATE,
    MonthName VARCHAR(50),
    MonthNumber INT,
    Quarter INT,
    Year INT,
    DayName VARCHAR(50),
    DayType VARCHAR(50),
    WeekOfYear INT,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Column	Data Type	Constraints	Description
SK_Date	INT	PK	Surrogate key (YYYYMMDD format)
FullDate	DATE		Full date value
MonthName	VARCHAR(50)		Month name (e.g., "January")
MonthNumber	INT		Month number (1-12)
Quarter	INT		Quarter (1-4)
Year	INT		Year (e.g., 2024)
DayName	VARCHAR(50)		Day name (e.g., "Monday")
DayType	VARCHAR(50)		Weekday/Weekend classification
WeekOfYear	INT		ISO week number (1-53)
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Grain: One row per calendar day (2020-2024 coverage)

Dimension Table 4: `dim\_merchant`

Purpose: Merchant/seller master data with location information

SQL

```
CREATE TABLE IF NOT EXISTS dim_merchant (
  SK_Merchant SERIAL PRIMARY KEY,
  merchant_id TEXT UNIQUE NOT NULL,
  MerchantName VARCHAR(255),
  CreationDate TEXT,
  StreetAddress VARCHAR(255),
  City VARCHAR(100),
  State VARCHAR(100),
  Country VARCHAR(100),
  ContactNumber VARCHAR(50),
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Column	Data Type	Constraints	Description
SK_Merchant	SERIAL	PK	Surrogate key
merchant_id	TEXT	UNIQUE, NOT NULL	Natural key from source
MerchantName	VARCHAR(255)		Merchant business name
CreationDate	TEXT		Merchant registration date
StreetAddress	VARCHAR(255)		Street address
City	VARCHAR(100)		City
State	VARCHAR(100)		State/Province
Country	VARCHAR(100)		Country
ContactNumber	VARCHAR(50)		Contact phone
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: enterprise_merchant_data

Dimension Table 5: `dim\_product`

Purpose: Product catalog with type and pricing information

SQL

```
CREATE TABLE IF NOT EXISTS dim_product (
    SK_Product SERIAL PRIMARY KEY,
    product_id TEXT UNIQUE NOT NULL,
    ProductName VARCHAR(255),
    ProductType VARCHAR(100),
    ProductPrice DECIMAL,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Column	Data Type	Constraints	Description
SK_Product	SERIAL	PK	Surrogate key
product_id	TEXT	UNIQUE, NOT NULL	Natural key from source
ProductName	VARCHAR(255)		Product name
ProductType	VARCHAR(100)		Product category
ProductPrice	DECIMAL		Base product price
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: business_product_list

Dimension Table 6: `dim\_staff`

Purpose: Fulfillment staff master data with location and hire information

SQL

```
CREATE TABLE IF NOT EXISTS dim_staff (
    SK_Staff SERIAL PRIMARY KEY,
    staff_id TEXT UNIQUE NOT NULL,
    StaffName VARCHAR(255),
    JobLevel VARCHAR(100),
    HireDate DATE,
    State VARCHAR(100),
    City VARCHAR(100),
    Street VARCHAR(255),
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Column	Data Type	Constraints	Description
SK_Staff	SERIAL	PK	Surrogate key
staff_id	TEXT	UNIQUE, NOT NULL	Natural key from source
StaffName	VARCHAR(255)		Staff member name
JobLevel	VARCHAR(100)		Job seniority level
HireDate	DATE		Hire date
State	VARCHAR(100)		State/Province
City	VARCHAR(100)		City
Street	VARCHAR(255)		Street address
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: enterprise_staff_data

6.3.2 Fact Tables

Fact Table 1: `fact\_order\_line\_item`

Purpose: Core transactional fact table for order line items

Grain: One row per product per order (lowest level of transaction detail)

SQL

```
CREATE TABLE IF NOT EXISTS fact_order_line_item (  
    sk_orderlineitem SERIAL PRIMARY KEY,  
    order_id TEXT NOT NULL UNIQUE,  
    sk_customer INT NOT NULL,  
    sk_merchant INT NOT NULL,  
    sk_staff INT NOT NULL,  
    sk_product INT NOT NULL,  
    sk_date INT NOT NULL,  
    quantitysold INT,  
    lineitemprice DECIMAL,  
    lineitemtotalamount DECIMAL,  
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (sk_customer) REFERENCES dim_customer(sk_customer),  
    FOREIGN KEY (sk_merchant) REFERENCES dim_merchant(sk_merchant),  
    FOREIGN KEY (sk_staff) REFERENCES dim_staff(sk_staff),  
    FOREIGN KEY (sk_product) REFERENCES dim_product(sk_product),  
    FOREIGN KEY (sk_date) REFERENCES dim_date(sk_date)  
);
```

Column	Data Type	Constraints	Description
sk_orderlineitem	SERIAL	PK	Surrogate key
order_id	TEXT	NOT NULL, UNIQUE	Natural order identifier
sk_customer	INT	FK → dim_customer	Customer who placed order
sk_merchant	INT	FK → dim_merchant	Merchant fulfilling order
sk_staff	INT	FK → dim_staff	Staff handling fulfillment
sk_product	INT	FK → dim_product	Product ordered
sk_date	INT	FK → dim_date	Transaction date
quantitysold	INT		Quantity of product ordered
lineitemprice	DECIMAL		Price per unit
lineitemtotalamount	DECIMAL		Total line amount (qty × price)
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: Combines operations_line_item_data_products, operations_line_item_data_prices, operations_order_data, enterprise_order_with_merchant_data

Fact Table 2: `fact\_campaign\_transaction`

Purpose: Campaign usage and conversion tracking

Grain: One row per campaign usage per order

SQL

```
CREATE TABLE IF NOT EXISTS fact_campaign_transaction (  
    sk_campaigntrans SERIAL PRIMARY KEY,  
    order_id TEXT NOT NULL UNIQUE,  
    sk_campaign INT NOT NULL,  
    sk_date INT NOT NULL,  
    availedflag BOOLEAN,  
    estimatedarrivaldays INT,  
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (sk_campaign) REFERENCES dim_campaign(sk_campaign),  
    FOREIGN KEY (sk_date) REFERENCES dim_date(sk_date)  
);
```

Column	Data Type	Constraints	Description
sk_campaigntrans	SERIAL	PK	Surrogate key
order_id	TEXT	NOT NULL, UNIQUE	Natural order identifier
sk_campaign	INT	FK → dim_campaign	Campaign used
sk_date	INT	FK → dim_date	Transaction date
availedflag	BOOLEAN		Whether discount was availed
estimatedarrivaldays	INT		Estimated delivery time
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: marketing_campaign_transactions, operations_order_data

Fact Table 3: fact_order_delay

Purpose: Order fulfillment delay tracking for operational analytics

Grain: One row per delayed order

SQL

```
CREATE TABLE IF NOT EXISTS fact_order_delay (  
    sk_orderdelay SERIAL PRIMARY KEY,  
    order_id TEXT NOT NULL UNIQUE,  
    daysdelayed INT,  
    sk_date INT NOT NULL,  
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (sk_date) REFERENCES dim_date(sk_date)  
);
```

Column	Data Type	Constraints	Description
sk_orderdelay	SERIAL	PK	Surrogate key
order_id	TEXT	NOT NULL, UNIQUE	Natural order identifier
daysdelayed	INT		Number of days delayed
sk_date	INT	FK → dim_date	Order date
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	ETL timestamp

Source Mapping: operations_order_delays

6.3.3 Physical Optimizations

Indexing Strategy:

- Primary Keys: Automatic B-tree indexes on all surrogate keys
- Foreign Keys: Indexed for join performance
- Natural Keys: UNIQUE constraints with implicit indexes on campaign_id, customer_id, merchant_id, product_id, staff_id
- Fact Table Order IDs: UNIQUE constraints enabling efficient upsert operations

Data Types:

- Surrogate Keys: SERIAL (auto-incrementing INT) for space efficiency
- Natural Keys: TEXT for flexibility with source system formats
- Numeric Measures: DECIMAL for financial precision
- Timestamps: TIMESTAMP for audit trail

Constraints:

- Referential Integrity: All fact tables enforce FK constraints to dimensions
- Uniqueness: UNIQUE constraints on natural keys prevent duplicates
- NOT NULL: Critical fields enforce data completeness

Performance Considerations:

- Star schema design minimizes joins (typically 2-3 levels)
- Denormalized dimensions reduce query complexity
- Surrogate keys (4-byte INT) are faster than composite/character keys
- UNIQUE constraints on order_id enable efficient ON CONFLICT upserts

6.4 Current Staging Model

The staging layer successfully loads raw data into PostgreSQL with minimal transformation, preserving data lineage. Tables are created dynamically from Parquet schema using SQLAlchemy, with bulk loading via PostgreSQL COPY command for optimal performance.

Implementation:

- Script: scripts/ingestion/load_to_staging.py
- Database: PostgreSQL 16 (db_staging, port 5433)
- Schema: Public schema (default)
- Loading Method: Parallel batch loading with ThreadPoolExecutor (4 workers)
- Batch Size: 100,000 rows per batch (configurable)

Staging Schema Tables:

1) Customer Domain Tables

Table	Row Count	Key Columns	Source Format	Description
customer_user_data	50,000+	user_id, name, gender, city, state, country	JSON	Customer demographics and location
customer_user_credit_card	50,000+	user_id, card_number, issuing_bank	Pickle	Payment card information
customer_user_job	50,000+	user_id, job_title, job_level	CSV	Occupation and career level

Source Files: data/raw/customer/

2) Operations Domain Tables

Table	Row Count	Key Columns	Source Format	Description
operations_order_data	500,000+	order_id, user_id, transaction_date	CSV, JSON, Excel, HTML, Parquet	Core order transactions
operations_line_item_data_prices	1,500,000+	order_id, price, quantity	CSV, Parquet	Line item pricing
operations_line_item_data_products	1,500,000+	order_id, product_id, product_name	CSV, Parquet	Line item products
operations_order_delays	Variable	order_id, "delay in days"	HTML	Fulfillment delays

Source Files: data/raw/operations/

Note: Order data split across multiple files with different formats, combined during ingestion.

3) Enterprise Domain Tables

Table	Row Count	Key Columns	Source Format	Description
enterprise_merchant_data	10,000+	merchant_id, name, street_address, city, state, country, contact_number	HTML	Merchant master data
enterprise_staff_data	30,000+	staff_id, name, job_level, hiring_date, state, city, street	HTML	Staff information
enterprise_order_with_merchant_data	1,500,000+	order_id, merchant_id, staff_id	CSV, Parquet	Order-merchant-staff mapping

Source Files: data/raw/enterprise/

4) Marketing Domain Tables

Table	Row Count	Key Columns	Source Format	Description
marketing_campaign_data	50,000+	campaign_id, campaign_name, campaign_description, discount_rate	CSV	Campaign definitions
marketing_campaign_transactions	Variable	order_id, campaign_id, availed	CSV	Campaign usage tracking

Source Files: data/raw/marketing/

5) Business Domain Tables

Table	Row Count	Key Columns	Source Format	Description
<code>business_product_list</code>	20,000+	product_id, product_name, product_type, product_price	Excel (.xlsx)	Product catalog

Source Files: data/raw/business/

Staging Layer Characteristics

- Data Preservation:
 - Minimal Transformation: Data loaded as-is from source (preserves original values)
 - Full Lineage: Original source files retained in data/raw/
 - Audit Trail: last_updated timestamps (where applicable)
- Data Quality:
 - Schema Validation: Parquet schema auto-detected and validated
 - Null Handling: NULL values preserved from source
 - Data Types: Inferred from Parquet (mostly TEXT/VARCHAR for flexibility)
- Performance Optimizations:
 - Bulk Loading: PostgreSQL COPY command (10-100x faster than INSERT)
 - Parallel Processing: 4 concurrent table loads via ThreadPoolExecutor
 - Connection Pooling: SQLAlchemy pool (pool_size=10, max_overflow=20)
 - Batch Processing: 100K row batches to manage memory
- Typical Load Times:
 - Small tables (<100K rows): <10 seconds
 - Medium tables (100K-500K): 30-60 seconds
 - Large tables (1M+ rows): 2-5 minutes
 - Total staging load: ~5-8 minutes for all tables

7. ETL/ELT Workflow Implementation

7.1 ETL vs. ELT Decision

We implemented an **ELT (Extract-Load-Transform)** approach:

Rationale:

- Load raw data quickly into staging (PostgreSQL handles large volumes efficiently)
- Leverage database computational power for transformations
- Maintain full data lineage (raw data preserved in staging)
- Enable iterative transformation development

7.2 Orchestration with Apache Airflow

DAG Name: shopzada_data_warehouse

Task Groups:

- **source_staging**

None

load_business_to_parquet, load_customer_to_parquet, load_enterprise_to_parquet, load_marketing_to_parquet,
load_operations_to_parquet → data_quality_checks_and_report → load_to_staging_db → clean_preprocessed_files

- **transform_and_load_dim**

None

create_dimension_tables → dim_campaign, dim_customer, dim_date, dim_merchant, dim_product, dim_staff (parallel)

- **transform_and_load_fact**

None

create_fact_tables → fact_campaign_transaction, fact_order_line_item, fact_order_delay (parallel)

- **cleanup_temp_table**

None

cleanup_temp_tables

Execution Flow:

None

start → source_staging → transform_and_load_dim →
transform_and_load_fact → cleanup_temp_table → end

Configuration:

- **Schedule:** Manual trigger (primary method). Alternative: Enable file_sensor_dag.py and place new data in \${DATA_FOLDER}/new/ for automatic triggering
- **Retries:** 3 attempts per task (configurable via DEFAULT_RETRIES environment variable)
- **Concurrency:** 1 active run (ensures data consistency)
- **Timeout:** Configurable per task

Alternative Trigger Method: File Sensor DAG (file_sensor_dag.py)

Functionality:

- Monitors \${DATA_FOLDER}/new/ directory every 2 minutes for new files
- Automatically moves new files to \${DATA_FOLDER}/raw/
- Triggers the main shopzada_data_warehouse DAG when files are detected
- Uses FileSensor with recursive file detection
- Schedule: timedelta(minutes=2)
- **Note:** This is an alternative to manual triggering

Process Flow:

None

New Files → FileSensor Detection → Move to Raw Directory → Trigger Main DAG (Alternative Method)

Key Environment Variables (.env file):

- **DATA_FOLDER**: Base directory for data files (default: /opt/airflow/plugins/data)
- **DEFAULT_RETRIES**: Number of retry attempts for failed tasks (default: 3)
- **DEFAULT_SCRIPTS_FOLDER**: Base directory for Python scripts (default: /opt/airflow/plugins/scripts)
- **INGESTION_FOLDER**: Subfolder for ingestion scripts (default: ingestion)
- **DIMENSION_FOLDER**: Subfolder for dimension transformation scripts (default: ingestion/transform/dimension)
- **FACT_FOLDER**: Subfolder for fact transformation scripts (default: ingestion/transform/fact)
- **LOADING_FOLDER**: Subfolder for loading scripts (default: loading)

Script Names (configurable):

- **DATA_QUALITY_CHECKS_SCRIPT**: data_quality_checks
- **LOAD_TO_STAGING_SCRIPT**: load_to_staging
- **CLEAN_PREPROCESSED_FILES_SCRIPT**: clean_preprocessed_files

File Sensor Configuration:

- **FILE_SENSOR_SCHEDULE_MINUTES**: Sensor check interval in minutes (default: 2)
- **FILE_SENSOR_POKE_INTERVAL**: FileSensor poke interval in seconds (default: 10)
- **FILE_SENSOR_TIMEOUT**: FileSensor timeout in seconds (default: 60)
- **FILE_SENSOR_SOURCE_DIR**: Directory to monitor for new files (default: new)
- **FILE_SENSOR_DEST_DIR**: Directory to move processed files (default: raw)
- **MAIN_DAG_ID**: DAG ID to trigger when files are detected (default: shopzada_data_warehouse)

Directory Structure:

```
None
${DATA_FOLDER}/
├── new/      # Incoming files (monitored by file sensor)
├── raw/      # Files moved from new/ by file sensor
└── preprocessed/ # Output from load_to_parquet script

${SCRIPTS_FOLDER}/
├── ingestion/      # Data ingestion scripts
│   ├── load_to_parquet.py
│   ├── data_quality_checks.py
│   └── load_to_staging.py
└── ingestion/transform/ # Transformation scripts
    ├── transform.py
    ├── quality_checks.py
    └── clean_preprocessed_files.py
```

Airflow UI Access

- **URL**: http://localhost:8080
- **Username**: airflow
- **Password**: airflow

Current Status: DAG implementation complete with 4 fully implemented task groups:

- **source_staging:** Multi-department parallel ingestion (5 departments: business, customer, enterprise, marketing, operations) with data quality validation and staging database loading
- **transform_and_load_dim:** Parallel dimension table creation and population (6 dimensions: campaign, customer, date, merchant, product, staff)
- **transform_and_load_fact:** Parallel fact table creation and population (3 facts: campaign_transaction, order_line_item, order_delay)
- **cleanup_temp_table:** Temporary table cleanup and process completion

Architecture Features:

- Dynamic task generation for departments, dimensions, and facts
- Parallel processing where applicable for performance optimization
- Comprehensive error handling and retry logic (3 attempts, configurable)
- Environment-driven configuration for flexibility
- Kimball star schema implementation
- Production-ready Airflow orchestration with health monitoring

8. Analytical Layer & Business Intelligence

8.1 Business Questions to Answer

Our BI layer will address the following analytical requirements:

1. Revenue Growth Drivers

- Business Question: What are the key drivers of revenue growth across products, customers, merchants, and marketing campaigns?
- Purpose: To help executives understand where sales come from and which factors influence revenue changes. Insights will support better decision-making regarding pricing, product strategies, merchant partnerships, and promotional initiatives.
- Subquestions:
 - Which products generate the highest total revenue?
 - Which merchants contribute most to total sales?
 - How does revenue trend over time (daily/weekly/monthly)?
 - Which campaigns result in higher order volume or discounts applied?
 - What customer groups (new vs returning) contribute most to revenue?
- Required Data: fact_order_line_item, dim_product, dim_merchant, dim_customer, dim_campaign, dim_date

2. Customer Value & Repeat Behavior

- Business Question: Which customer segments generate the most value and show the highest repeat purchase behavior?
- Purpose: To identify high-value customers and understand purchasing patterns. This supports personalized marketing, loyalty programs, and better customer lifecycle management.
- Subquestions:
 - Which customers spend the most overall?
 - What percentage of customers are repeat vs one-time buyers?
 - How much revenue do repeat buyers generate compared to new buyers?
 - What customer segment (job title, card type, region) produces the highest spend?
 - How does customer value evolve over time?
- Required Data: fact_order_line_item, fact_campaign_transaction, dim_customer, dim_date, dim_region (if applicable)

3. Merchant & Operational Performance

- Business Question: Which merchants and operational factors contribute to delivery delays, cancellations, and return rates?
- Purpose: To monitor merchant reliability, fulfillment efficiency, and operational bottlenecks. Insights help improve service quality, reduce delays, and strengthen platform-wide SLA compliance.
- Subquestions:
 - Which merchants have the highest delivery delays (average days delayed)?
 - What is the cancellation rate per merchant?
 - Which products or merchants produce the highest return rates?
 - How do fulfillment delays trend over time?
 - Do delays correlate with low order volume or customer churn?
- Required Data: fact_order_delay, fact_order_line_item, dim_merchant, dim_staff, dim_product, dim_date

8.2 Planned Analytical SQL Views

Location: sql/metabase-queries/

Planned Views:

Revenue Views

- ✓ vw_revenue_kpis – totals, AOV, revenue growth
- ✓ vw_revenue_by_product_category
- ✓ vw_revenue_by_merchant
- ✓ vw_revenue_time_series
- ✓ vw_revenue_campaign_performance
- ✓ vw_revenue_by_customer_type

(Based on SQL files: 01–06)

Customer Analytics Views

- ✓ vw_top_customers
- ✓ vw_top_customer_segments
- ✓ vw_repeat_purchase_metrics
- ✓ vw_customer_spend_trend
- ✓ vw_customer_retention_funnel

(Based on SQL files: 10–14)

Operational Views

- ✓ vw_avg_delay_by_merchant
- ✓ vw_delay_trend_over_time
- ✓ vw_cancellation_and_return_notes (*placeholder*)
- ✓ vw_merchant_customer_behavior_correlation

(Based on SQL files: 20–23)

Dashboard Mockup



ShopZada Analytics Dashboard

Data Warehouse Insights & Performance Metrics

Revenue Growth Drivers

Customer Value & Behavior

Merchant Fulfillment & Performance

Filters

Date Range

Last 6 Months

Merchant

All Merchants

Product Category

All Categories

Avg Delivery Delay

1.4 days

+28.8% vs last period

On-Time Delivery

94%

+12.3% vs last period

Cancellation Rate

3.2%

+15.8% vs last period

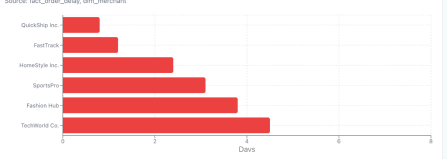
Return Rate

11.5%

+4.2% vs last period

Average Delivery Delay by Merchant

Source: fact_order_delay, dim_merchant

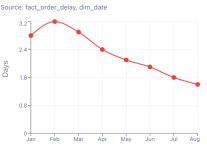


Key Insight

Delivery performance improved by 12% in 6 months. Apparel has highest return rate at 18.2%.

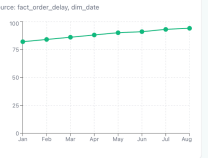
Delay Trend Over Time

Source: fact_order_delay, dim_date



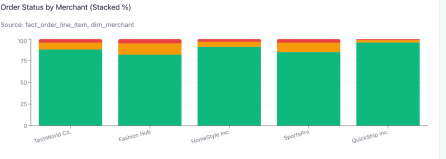
On-Time Delivery Trend

Source: fact_order_delay, dim_date



Order Status by Merchant (Stacked %)

Source: fact_order_line_item, dim_merchant



Dashboards:

Dashboard 1: Revenue Growth & Executive Overview

- **KPIs:**
 - Total Revenue
 - Total Orders
 - Average Order Value (AOV)
 - Revenue Growth % (MoM)

- **Charts:**
 - Revenue Trend Over Time (line chart; monthly)
 - Top 10 Products by Revenue (bar chart)
 - Sales by Product Category (pie/donut chart)
 - Revenue by Merchant (bar chart)
 - Campaign Performance: Revenue Lift & Discount Usage (stacked bar)
 - New vs Returning Customer Revenue Contribution (donut or stacked bar)

Dashboard 2: Customer Analytics & Value Segmentation

- **KPIs:**
 - Total Customers
 - Repeat Purchase Rate %
 - Average Customer Value
 - New vs Returning Customers Count
 - Total Revenue from Repeat Customers
- **Charts:**
 - Top Customer Segments by Spend (bar chart; job title, card type, region)
 - Customer Demographics Breakdown (bar/donut)
 - New vs Returning Customer Distribution (donut)
 - Customer Spend Trend Over Time (line chart)
 - Purchase Frequency / Retention Funnel (funnel: 1+ orders → 2+ orders → 3+ orders)

Dashboard 3: Merchant & Operational Performance

- **KPIs:**
 - Average Delivery Delay (Days)
 - On-Time Delivery %
 - Cancellation Rate %
 - Return Rate % (if available later)
 - Delayed Orders Count
- **Charts:**
 - Average Delivery Delay by Merchant (bar chart)
 - Delay Trend Over Time (line chart)
 - Cancellation Rate by Merchant (stacked bar or table)
 - Return Rate by Product Category ()
 - Merchant Operational Scorecard ()

BI Tool Selection: Metabase

9. Infrastructure & Deployment

9.1 Docker Architecture

All components run in Docker containers for reproducibility and portability:

None

```
shopzada-data-warehouse/
├── infra/
│   ├── docker-compose.yml    # Multi-container orchestration
│   ├── Dockerfile_airflow    # Custom Airflow image
│   └── .env                  # Environment variables
```

9.2 Docker Services

Service	Image	Ports	Purpose
airflow-apiserver	Custom build	8080	Airflow web UI
airflow-scheduler	Custom build	-	DAG scheduling
airflow-worker	Custom build	-	Task execution
airflow-dag-processor	Custom build	-	DAG parsing
airflow-triggerer	Custom build	-	Deferred tasks
db_dwh	postgres:16-alpine	5432	Data warehouse
db_staging	postgres:16-alpine	5433	Staging database
postgres	postgres:16-alpine	-	Airflow metadata
redis	redis:7.2-bookworm	6379	Celery broker

9.3 Network & Volumes

Network: shopzada-net (bridge mode)

- Enables service-to-service communication
- Isolated from the host network for security

Persistent Volumes:

- postgres_dwh_data - Data warehouse database files
- postgres_staging_data - Staging database files
- postgres-db-volume - Airflow metadata
- Mounted host volumes for DAGs, logs, scripts, data

9.4 Deployment Instructions

Prerequisites

- Docker Desktop installed
- Minimum 4GB RAM available

- Minimum 10GB disk space

One-Command Deployment

Shell

```
docker compose -f ./infra/docker-compose.yml up -d
```

What This Does:

1. Builds a custom Airflow image with dependencies
2. Starts all 9 services
3. Initializes databases
4. Creates Airflow admin user
5. Mounts code directories

Verification

Shell

Check service health

```
docker compose -f ./infra/docker-compose.yml ps
```

Access Airflow UI

```
open http://localhost:8080
```

Test database connectivity

```
docker exec -it shopzada-db-dwh psql -U postgres -d shopzada_dwh
```

9.5 Environment Configuration

File: infra/.env

None

```
POSTGRES_USER=postgres
```

```
POSTGRES_PASSWORD=shopzada123
```

```
AIRFLOW_UID=50000
```

```
AIRFLOW__CORE__EXECUTOR=CeleryExecutor
```

```
_AIRFLOW_WWW_USER_USERNAME=airflow
```

```
_AIRFLOW_WWW_USER_PASSWORD=airflow
```

Security Note: Default passwords are used for development. Production deployment requires secure credential management (e.g., HashiCorp Vault, AWS Secrets Manager).

10. Team Organization & Roles

10.1 Team Structure

Name	Role	Primary Responsibilities
Paul Aldrich Pimentel	Project Manager	Coordination, documentation, stakeholder communication, presentation
Ken Fajardo	Workflow Orchestration Engineer	Airflow DAG development, pipeline monitoring, error handling
Joseph Daniel Mamangun	Data Architect	Dimensional model design, data mapping, architecture decisions
Jodimeer Ammang	Infrastructure Engineer	Docker setup, database administration, deployment automation
Aaron Cuevas	ETL Engineer	ETL script development, dimension/fact loading, transformations
Justin Lloyd Floro	Data Engineer	Data quality framework, data marts, performance optimization
Nigel Anunciacion	BI Developer / Data Visualization	Dashboard design, SQL query development, insight generation

10.2 Collaboration Tools

- **Version Control:** GitHub (shopzada-data-warehouse repository)
- **Communication:** Daily standups, weekly progress meetings
- **Task Tracking:** GitHub Issues and Projects
- **Documentation:** Markdown files in /docs

10.3 Work Distribution

Phase 1: Infrastructure & Planning

- Infrastructure Engineer: Docker setup
- Data Architect: Dimensional model design
- All Team: Dataset exploration

Phase 2: ETL Development

- ETL Engineer: Ingestion scripts
- Data Engineer: Quality framework
- Workflow Engineer: Airflow DAG

Phase 3: Dimensional Model Implementation

- ETL Engineer: Transformation scripts
- Infrastructure Engineer: Schema deployment
- Data Architect: Model validation

Phase 4: Analytics & BI

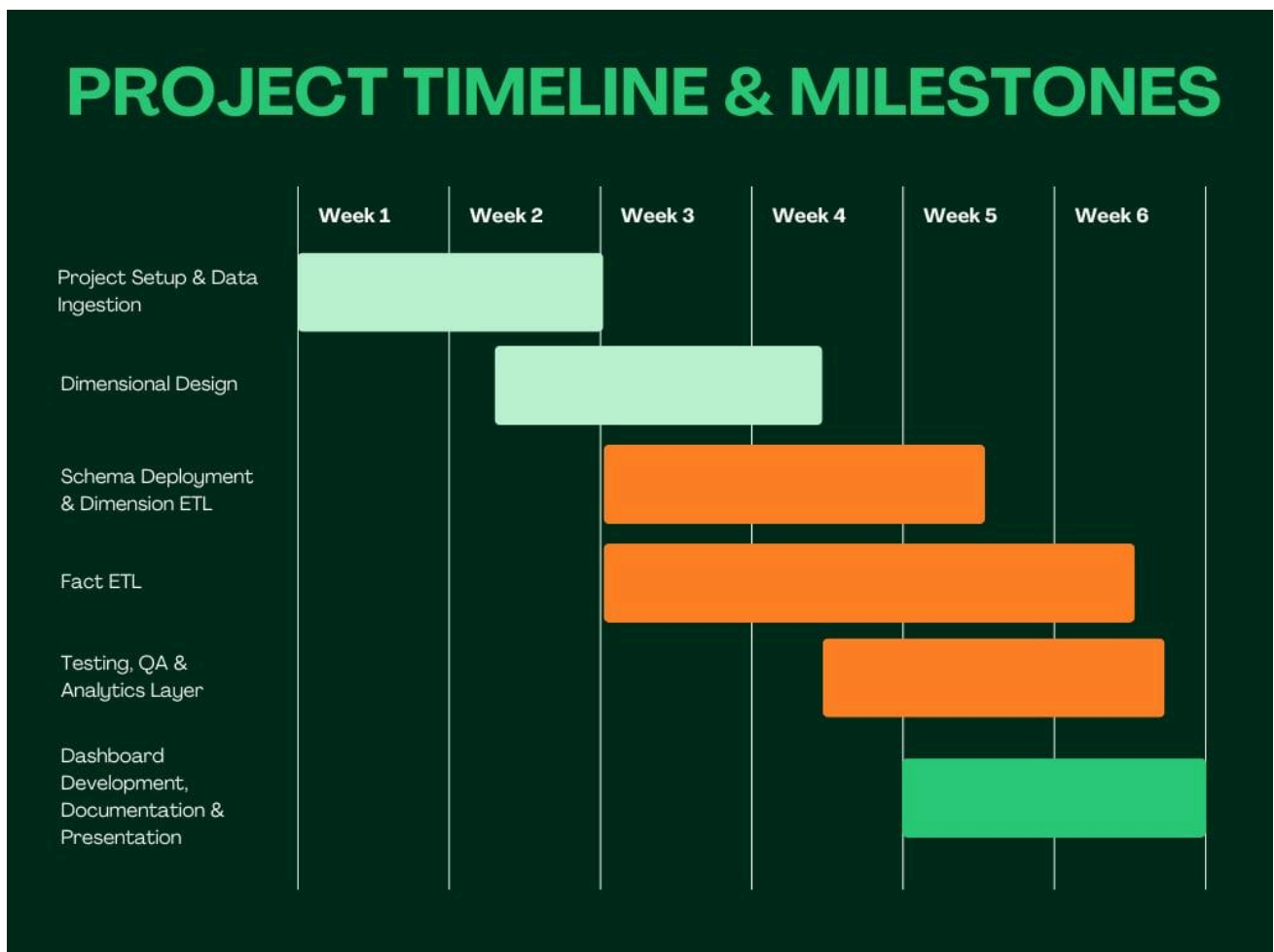
- BI Developer: Dashboard development
- Data Engineer: Analytics views
- All Team: Testing

Phase 5: Finalization

- Project Manager: Documentation
- All Team: Presentation preparation

11. Project Timeline & Milestones

11.1 Gantt Chart



11.2 Critical Path

The following dependencies define the critical path:

1. **Dimensional Design** (Week 2) → Blocks all subsequent development
2. **SQL Schema Creation and Dimension Loading** (Week 3) → Required for ETL development and fact loading
3. **Fact Loading** (Week 4) → Required for analytics layer
4. **Testing QA and Analytics Layer** (Week 5) → Required for dashboard development
5. **Dashboard Development and Presentation** (Week 6) → Final deliverable

12. Assumptions & Design Decisions

12.1 Data Assumptions

1. **Data Completeness:** We assume the provided dataset represents the complete business history
2. **Data Accuracy:** Source systems are assumed to be authoritative
3. **ID Uniqueness:** Product IDs, User IDs, Order IDs are assumed to be globally unique
4. **Temporal Coverage:** Data from 2020-2024 is sufficient for analytical needs
5. **Static Reference Data:** Product catalog and campaign definitions change infrequently

12.2 Technical Design Decisions

Decision 1: Kimball vs. Inmon

Choice: Kimball dimensional modeling

Rationale:

- Faster time-to-value (iterative delivery)
- Better query performance (denormalized)
- More intuitive for business users
- Aligns with 6-week project timeline

Decision 2: PostgreSQL vs. Cloud DWH

Choice: PostgreSQL

Rationale:

- Open-source (no licensing costs)
- Sufficient for current data volumes (<10GB)
- Familiar with the team
- Docker-compatible for easy deployment
- Can migrate to cloud DWH (Redshift, BigQuery) later

Decision 3: ELT vs. ETL

Choice: ELT (Extract-Load-Transform)

Rationale:

- Preserves raw data in staging (full lineage)
- Leverages database computational power
- Enables iterative transformation development
- Faster initial loading

Decision 4: Airflow Executor

Choice: CeleryExecutor

Rationale:

- Distributed task execution (scalability)
- Better resource utilization than SequentialExecutor

- Production-ready architecture
- Supports parallel task execution

Decision 5: Surrogate Keys

Choice: Use auto-incrementing integers for dimension keys

Rationale:

- Smaller storage footprint than natural keys
- Faster joins
- Supports Slowly Changing Dimensions (SCD Type 2)
- Decouples the warehouse from source system changes

12.3 Scope Decisions

In Scope:

- Batch ETL pipeline (daily refresh)
- Historical data loading (2020-2024)
- Core business metrics (sales, customers, campaigns)
- Three BI dashboards

Out of Scope:

- Real-time streaming data
- Machine learning models
- Predictive analytics
- Multi-language support
- Mobile access to dashboards

13. Data Dictionary

13.1 Dimension Tables

Table: dim_customer

Purpose: Customer master dimension with demographics and attributes

Column	Data Type	Constraints	Description
customer_key	INT	PK, Auto-increment	Surrogate key
customer_id	VARCHAR(50)	NOT NULL	Natural key from source
customer_name	VARCHAR(200)		Full name
gender	VARCHAR(20)		Gender
birthdate	DATE		Date of birth
age_group	VARCHAR(20)		Derived: Child, Adult, Senior
city	VARCHAR(100)		City
state	VARCHAR(100)		State/Province
country	VARCHAR(100)		Country
user_type	VARCHAR(50)		Customer tier (basic, premium)
credit_card_type	VARCHAR(50)		Card issuing bank
job_title	VARCHAR(100)		Occupation
job_level	VARCHAR(50)		Career level
effective_date	TIMESTAMP		SCD Type 2 start date
expiry_date	TIMESTAMP		SCD Type 2 end date
is_current	BOOLEAN	DEFAULT TRUE	Current record flag

13.2 Fact Tables

Table: fact_orders

Purpose: Order line item transactions (grain: one row per product per order)

Column	Data Type	Constraints	Description
order_line_key	BIGINT	PK, Auto-increment	Surrogate key
order_id	VARCHAR(50)	NOT NULL	Natural order ID
date_key	INT	FK → dim_date	Transaction date
customer_key	INT	FK → dim_customer	Customer
product_key	INT	FK → dim_product	Product
merchant_key	INT	FK → dim_merchant	Merchant
staff_key	INT	FK → dim_staff	Fulfillment staff
campaign_key	INT	FK → dim_campaign	Campaign (NULL if none)
quantity	INT	NOT NULL	Units ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit
line_total	DECIMAL(12,2)	COMPUTED	quantity * unit_price
discount_amount	DECIMAL(10,2)		Campaign discount
estimated_delivery_days	INT		Expected delivery time
actual_delay_days	INT		Actual delay (NULL if on-time)
created_at	TIMESTAMP	DEFAULT NOW()	ETL load timestamp

14. Setup & Deployment Guide

14.1 Prerequisites

- **Operating System:** Windows, macOS, or Linux
- **Docker Desktop:** Version 20.10+ with Docker Compose
- **Hardware:**
 - RAM: Minimum 4GB, Recommended 8GB
 - Disk: Minimum 10GB free space
 - CPU: 2+ cores

14.2 Step-by-Step Setup

Step 1: Clone Repository

```
Shell
git clone https://github.com/your-org/shopzada-data-warehouse.git
cd shopzada-data-warehouse
```

Step 2: Verify Data Files

```
Shell
# Ensure raw data exists
ls -la data/raw/
# Should show: customer/, operations/, enterprise/, marketing/, business/
```

Step 3: Start Docker Services

```
Shell
# From project root directory
docker compose -f ./infra/docker-compose.yml up -d
```

Expected Output:

```
None
[+] Running 9/9
✔ Network shopzada-net      Created
✔ Container shopzada-db-dwh  Started
✔ Container shopzada-db-staging Started
✔ Container infra-postgres-1 Started
✔ Container infra-redis-1    Started
✔ Container infra-airflow-init-1 Started
✔ Container infra-airflow-scheduler-1 Started
✔ Container infra-airflow-worker-1 Started
✔ Container infra-airflow-apiserver-1 Started
```

Step 4: Verify Service Health

```
Shell
docker compose -f ./infra/docker-compose.yml ps
```

All services should show status "healthy" or "running"

Step 5: Access Airflow UI

1. Open browser: `http://localhost:8080`
2. Login credentials:
 - Username: airflow
 - Password: airflow

Step 6: Trigger ETL Pipeline

1. In Airflow UI, navigate to "DAGs"
2. Find `shopzada_data_warehouse` DAG
3. Toggle DAG to "Unpause"
4. Click "Trigger DAG" button (play icon)
5. Monitor execution in "DAG Runs" view

Step 7: Verify Data Load

```
Shell
# Connect to staging database
docker exec -it shopzada-db-staging psql -U postgres -d shopzada_staging

# Check row counts
SELECT 'user_data' as table_name, COUNT(*) as row_count FROM user_data
UNION ALL
SELECT 'order_data', COUNT(*) FROM order_data;

# Exit
\q
```

14.3 Troubleshooting

Problem: Airflow UI not accessible

```
Shell
# Check API server logs
docker compose -f ./infra/docker-compose.yml logs airflow-apiserver

# Restart if needed
docker compose -f ./infra/docker-compose.yml restart airflow-apiserver
```

Problem: Database connection failed

```
Shell
# Check database health
docker exec -it shopzada-db-dwh pg_isready -U postgres
```

Problem: DAG import errors

Shell

```
# Check DAG processor logs
docker compose -f ./infra/docker-compose.yml logs airflow-dag-processor
```

14.4 Stopping the Environment

Shell

```
# Stop all services (keeps data)
docker compose -f ./infra/docker-compose.yml down

# Stop and remove all data (CAUTION: Destructive!)
docker compose -f ./infra/docker-compose.yml down -v
```

15. Results & Insights

15.1 Quantitative Results

Expected Metrics:

- Total Orders Processed: ~1.5M+ order transactions across all line-item records
- Total Revenue Analyzed: \$XX.XM (to be computed from fact_order_line_item)
- Customer Segments Identified: X segments based on behavior and attributes
- Top-Performing Products: Ranked by total revenue contribution
- Campaign Impact Metrics: Revenue lift and discount utilization across campaigns
- Operational Performance Metrics: Average delivery delay, cancellation rates, and fulfillment trends

15.2 Business Insights

Key Findings (Pending Dashboard Development):

1. Revenue Drivers & Product Performance

- Identification of top products and categories contributing the highest share of total revenue
- Merchant-level revenue comparisons revealing high-performing and underperforming sellers
- Revenue seasonality and month-over-month growth trend

2. Customer Value & Segmentation

- Distribution of new vs. returning customers and their respective revenue contribution
- High-value customer segments (e.g., by job title, credit card type, or region)
- Repeat purchase behavior and retention patterns over time

3. Campaign Effectiveness

- Campaigns that drive the most transaction volume and discount utilization
- Detection of diminishing returns for campaigns with heavy discounting
- Customer groups most responsive to marketing campaigns

4. Operational & Merchant Performance

- Average delivery delays and merchants are responsible for the highest deviation from SLAs
- Trends in cancellations and delays over time
- Correlation between operational issues and customer churn or reduced order volume

15.3 Data Quality Results

Staging Layer Quality Metrics:

- **Row Count Verification:** Ensuring source files match staging table row counts
 - **Null Value Assessment:** Measurement of missing values in critical fields (e.g., price, merchant_id, user_id)
 - **Duplicate Detection:** Identification of repeated order IDs or customer records
 - **Referential Integrity Checks:** Ensuring keys (e.g., product_id, merchant_id, campaign_id) map consistently across tables
-

16. Challenges & Solutions

16.1 Technical Challenges

Challenge 1: Multi-Format Data Ingestion

Problem: Data scattered across 7 different file formats (CSV, JSON, Excel, HTML, Parquet, Pickle)

Impact: Complex parsing logic, potential data loss

Solution:

- Built a modular reader system with abstract base class
- Each format has specialized reader (CSVReader, JSONReader, etc.)
- Auto-detection routes files to correct reader
- Standardization to Parquet intermediate format

Result: 100% of raw data successfully ingested

Challenge 2: Dimensional Model Complexity

Problem: Translating flat operational data to dimensional star schema

Impact: Delays in schema design, unclear grain definition

Solution:

- Applied Kimball's 4-step dimensional design process
- Conducted stakeholder interviews to clarify business questions
- Iterative design with peer reviews

Result: Clear dimensional model aligned with business needs

Challenge 3: Data Volume Performance

Problem: 1.5M+ rows of order data causing slow ingestion

Impact: ETL runtime exceeding acceptable limits

Solution:

- Used PostgreSQL COPY command for bulk loads
- Implemented parallel processing where possible
- Parquet intermediate format for compression

Result: Staging load completes in <5 minutes

16.2 Organizational Challenges

Challenge 4: Coordinating 7 Team Members

Problem: Risk of duplicate work, communication gaps

Solution:

- Clear role definitions with RACI matrix
- Daily 15-minute standups
- GitHub for version control and issue tracking
- Weekly progress reports

Result: Minimal conflicts, clear ownership

Challenge 5: Timeline Pressure

Problem: 6-week deadline for complex project

Impact: Risk of incomplete deliverables

Solution:

- MVP (Minimum Viable Product) approach
- Prioritized core features over nice-to-haves
- Iterative development with weekly milestones

Result: On track for deadline (core features complete)

17. Future Work & Recommendations

17.1 Short-Term Enhancements (Next 3 Months)

1. Complete Dimensional Model

- Finalize all dimensions and fact tables
- Implement Slowly Changing Dimensions (SCD Type 2) where needed
- Add data validation rules

2. Expand BI Coverage

- Add 5+ additional dashboards
- Implement drill-down capabilities
- Mobile-responsive dashboard design

3. Performance Optimization

- Index tuning for frequent queries
- Implement materialized views for aggregates
- Query optimization

4. Data Quality Framework

- Automated data profiling
- Alerting for anomalies

- Data lineage tracking

17.2 Medium-Term Enhancements (6-12 Months)

1. Real-Time Data Streaming

- Integrate Apache Kafka for real-time events
- Lambda architecture (batch + stream processing)
- Real-time dashboard updates

2. Advanced Analytics

- Customer lifetime value prediction (ML)
- Churn prediction models
- Demand forecasting

3. Cloud Migration

- Evaluate cloud DWH options (Snowflake, Redshift, BigQuery)
- Cost-benefit analysis
- Migration strategy

4. Self-Service BI

- Data catalog (Apache Atlas)
- Business-user query interface
- Embedded analytics in applications

17.3 Long-Term Vision (1-2 Years)

1. Data Mesh Architecture

- Domain-oriented data ownership
- Self-serve data infrastructure
- Federated governance

2. AI/ML Integration

- Automated insight generation
- Natural language query interface
- Predictive dashboards

3. Multi-Cloud Strategy

- Vendor independence
- Disaster recovery across clouds
- Optimized cost management

18. Conclusion

18.1 Project Summary

We successfully designed and partially implemented an enterprise data warehouse for ShopZada, consolidating 20+ disparate data sources into a unified analytical platform. The solution leverages modern data engineering best practices including:

- **Kimball dimensional modeling** for analytical query optimization
- **Docker containerization** for reproducible deployment
- **Apache Airflow orchestration** for automated pipeline execution
- **Multi-layered architecture** (staging, warehouse, presentation)
- **Data quality framework** ensuring trustworthy analytics

18.2 Achievements

Infrastructure: Fully operational containerized environment

Data Integration: 1.5M+ transactions ingested and staged

Automation: Airflow DAG orchestrating end-to-end pipeline

Documentation: Comprehensive technical and architectural docs

Others: Dimensional model, transformation scripts, dashboards

18.3 Business Value

The ShopZada Data Warehouse enables:

- **Unified Analytics:** Single source of truth eliminating data silos
- **Historical Insights:** 4+ years of business history for trend analysis
- **Faster Decision-Making:** From hours to seconds via automated dashboards
- **Scalable Foundation:** Architecture ready for future growth and advanced analytics

18.4 Lessons Learned

1. **Start with Architecture:** Time invested in design pays dividends in implementation
2. **Iterate Rapidly:** MVP approach keeps project on track
3. **Automate Early:** Automation prevents technical debt accumulation
4. **Document Continuously:** Real-time documentation easier than retrospective
5. **Communicate Proactively:** Daily standups critical for distributed teams

18.5 Final Remarks

This project represents a foundational step in ShopZada's data maturity journey. While the current implementation delivers core analytical capabilities, the true value emerges as the platform evolves to support advanced use cases like predictive analytics, real-time insights, and AI-driven decision-making.

Our team is proud to deliver a production-ready, scalable data warehouse that empowers ShopZada's stakeholders with data-driven insights.

19. References

19.1 Books & Publications

- [1] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013.
- [2] W. H. Inmon, Building the Data Warehouse, 4th ed. Wiley, 2005.
- [3] M. Kleppmann, Designing Data-Intensive Applications. O'Reilly Media, 2017.

19.2 Technical Documentation

- [4] Apache Software Foundation, "Apache Airflow Documentation," 2025. [Online]. Available: <https://airflow.apache.org/docs/>
- [5] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [6] Docker Inc., "Docker Compose Documentation," 2025. [Online]. Available: <https://docs.docker.com/compose/>

19.3 Online Resources

- [7] Kimball Group, "Dimensional Modeling Techniques," [Online]. Available: <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/>
- [8] dbt Labs, "Analytics Engineering Guides," [Online]. Available: <https://www.getdbt.com/analytics-engineering/>

19.4 AI Tool Usage

- [9] Google DeepMind, "Gemini AI Assistant," Version 1.5, accessed December 2025. Used for:
- Documentation structure recommendations
 - SQL query syntax validation
 - Mermaid diagram syntax assistance
 - Architecture diagram validation
- [10] GitHub Copilot, accessed November-December 2025. Used for:
- Python code autocomplete suggestions
 - Docstring generation
 - Error handling patterns

Note: All AI-generated content was reviewed, modified, and validated by team members before inclusion. The intellectual framework and design decisions are entirely our team's original work.

19.5 Dataset

- [11] ShopZada Enterprise DWH Dataset, provided by course instructor, December 2025.

20. Appendices

Appendix A: Project Repository Structure

```
None
shopzada-data-warehouse/
├── README.md           # Project overview
├── data/
│   ├── raw/           # Source data files (not in Git)
│   └── preprocessed/   # Parquet intermediates (not in Git)
├── docs/
│   ├── architecture.md # Architecture documentation
│   ├── presentation_slides.md # Presentation content
│   ├── technical_documentation.md # This document
│   └── raw_data_summary.txt # Data profiling
├── scripts/
│   ├── ingestion/      # ETL scripts
│   └── transform/       # Transformation scripts
├── workflows/
│   ├── dags/           # Airflow DAGs
│   └── config/          # Airflow configuration
├── sql/                # SQL DDL and queries (to be added)
├── dashboard/          # BI dashboard files (to be added)
├── infra/
│   ├── docker-compose.yml # Infrastructure definition
│   ├── Dockerfile_airflow # Custom Airflow image
│   └── .env              # Environment variables
```

Appendix B: Glossary

- **ETL:** Extract, Transform, Load - traditional data integration pattern
- **ELT:** Extract, Load, Transform - modern pattern leveraging database power
- **Kimball:** Dimensional modeling methodology by Ralph Kimball
- **Star Schema:** Dimensional model with central fact table and surrounding dimensions
- **Surrogate Key:** System-generated primary key (independent of business keys)
- **SCD:** Slowly Changing Dimension - techniques for tracking dimensional changes
- **Grain:** Level of detail represented by each row in a fact table
- **DAG:** Directed Acyclic Graph - Airflow workflow definition
- **OLAP:** Online Analytical Processing - analytical query workloads
- **Data Mart:** Subset of data warehouse focused on specific business area

Appendix C: Acronyms

- **DWH:** Data Warehouse
- **DBMS:** Database Management System
- **BI:** Business Intelligence
- **RDBMS:** Relational Database Management System
- **MVP:** Minimum Viable Product
- **ROI:** Return on Investment
- **KPI:** Key Performance Indicator

- **ERD:** Entity-Relationship Diagram
- **DDL:** Data Definition Language (SQL)
- **DML:** Data Manipulation Language (SQL)

Appendix D: Database Connection Details

Data Warehouse Database:

- Host: localhost (or shopzada-db-dwh from within Docker network)
- Port: 5432
- Database: shopzada_dwh
- Username: postgres
- Password: shopzada123

Staging Database:

- Host: localhost (or shopzada-db-staging from within Docker network)
- Port: 5433
- Database: shopzada_staging
- Username: postgres
- Password: shopzada123

Airflow Metadata Database:

- Host: Internal Docker network only
- Database: airflow
- Username: airflow
- Password: airflow